

OS-SKILL 2

1. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop. To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

Building a simple shell — the main loop: display a prompt, read a command, and exit on "exit"

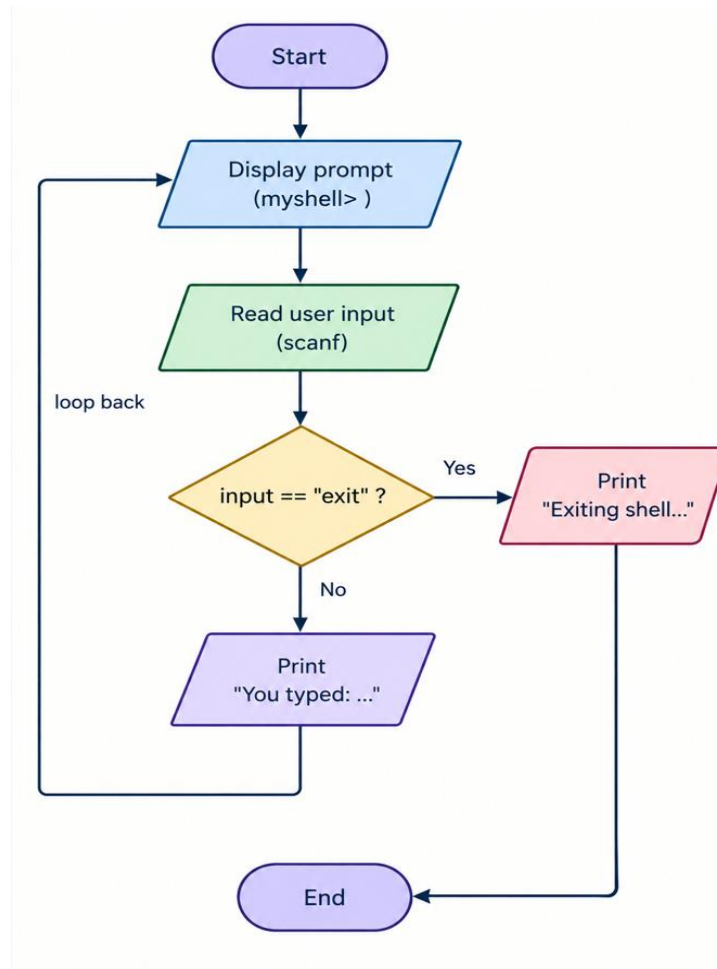
- Wrote the shell's main loop in `os_lab/skill2.c` using `gedit` — it displays a `myshell>` prompt, reads a command with `scanf`, and stops when the user types `exit`.
- Compiled the program with `gcc skill2.c` and started it with `./a.out` — the `myshell>` prompt appears, ready for input.
- Tested the shell: typed some sentence and date (each is echoed back), then `exit`, which prints "Exiting shell..." and ends the loop.

```
vida@LAPTOP-4B7UUAUH:~/os_lab$ gedit skill2.c
** (gedit:3872): WARNING **: 18:05:12.964: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:3872): WARNING **: 18:05:12.964: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
vida@LAPTOP-4B7UUAUH:~/os_lab$ gcc skill2.c
vida@LAPTOP-4B7UUAUH:~/os_lab$ ./a.out
```

```
vida@LAPTOP-4B7UUAUH:~/os_lab$ gcc skill2.c
vida@LAPTOP-4B7UUAUH:~/os_lab$ ./a.out
myshell> welcome to kl university
User has typed: welcome
myshell> User has typed: to
myshell> User has typed: kl
myshell> User has typed: university
myshell> operating_systems
User has typed: operating_systems
myshell> exit
Exiting shell...
vida@LAPTOP-4B7UUAUH:~/os_lab$ |
```

```
vida@LAPTOP-4B7UUAUH:~/os_lab$ ./a.out
myshell> kluniversity_courses
User has typed: kluniversity_courses
myshell> date
User has typed: date
myshell> exit
Exiting shell...
```

Control Flow Diagram



- The shell repeats one loop: show the prompt, read the input, then either exit (if the command is exit) or echo the command and loop back.